

PROJECT REPORT

Academic Reward Points & Student Performance Management System

A Full-Stack Web Platform for Gamified Academic Engagement

Team: Doom Developers

Event: TechNova Hackathon

Repository: github.com/Ukash-07/Doom_Developers_TechNova_Hackathon

Report Date: August 2026

Executive Summary

The Academic Reward Points & Student Performance Management System is a full-stack web application built for the TechNova Hackathon by team Doom Developers. It gives higher education institutions a structured way to incentivize, track, and evaluate student achievement across problem-solving practice, hackathon participation, and industry certifications. Achievements are converted into Reward Points (RP), which are in turn scaled into a bonus Internal Academic Mark out of 11, giving faculty a transparent, data-backed way to reward co-curricular effort alongside classroom performance.

The platform is organized around three roles — Admin, Faculty (assigned to a specific domain), and Student — each with a dedicated dashboard, and it is built on a React/Vite frontend, a Node.js/Express REST API backend, and a MySQL database with a JSON fallback for environments without a database server.

1. Problem Statement

Most academic institutions currently track a student's co-curricular and skill-building achievements — coding practice, hackathon wins, certifications, internships — through disconnected, manual processes: spreadsheets maintained by individual faculty, paper certificates submitted physically, and ad-hoc decisions about how much credit an achievement deserves. This creates several concrete problems that the project set out to solve:

- No single source of truth: a student's problem-solving progress, hackathon record, and certifications live in different files owned by different faculty, making institution-wide reporting slow and error-prone.
- Inconsistent and subjective credit: without a common point system, similar achievements are rewarded differently depending on which faculty member evaluates them.
- Manual, non-auditable verification: proof of achievement (certificates, GitHub links) is typically emailed or handed in, with no timestamped record of who submitted what, who approved it, and when.
- No visibility for students: students have limited ability to see their own standing relative to peers, track progress over time, or know which activities are currently available to boost their score.
- Disconnected from grading: even when co-curricular effort is informally rewarded, it is rarely converted into a transparent, formula-based contribution to a student's internal academic marks.
- Support requests get lost: questions or disputes about points raised by students to faculty have no ticketing or audit trail, so response times and accountability suffer.

2. Proposed Solution

The team designed a unified, role-based web platform that digitizes the entire reward lifecycle — from activity announcement, to student submission, to faculty verification, to automatic conversion into internal marks — and layers a leaderboard and analytics on top to keep students motivated.

2.1 Core Concept: Reward Points (RP)

Every academic or co-curricular achievement — problem-solving milestones, hackathon results, certifications — is assigned a Reward Point (RP) value by the responsible faculty domain lead. A student accumulates RP over the semester by completing posted activities and submitting proof, giving a single numeric measure of engagement that is comparable across the whole class.

2.2 Domain-Based Faculty Ownership

Rather than one faculty member trying to judge every type of achievement, responsibility is split across three specialized domain leads, so evaluation stays consistent and subject-matter-appropriate:

- PS Lead — problem-solving assessments, LeetCode milestones, and lab coding.
- Hackathon Lead — technical competitions, coding hackathons, and paper presentations.
- Certifications Lead — NPTEL courses, AWS/Cloud certifications, and industrial internships.

2.3 Transparent Conversion to Internal Marks

Instead of leaving the RP-to-marks conversion to individual discretion, the system applies one institution-wide formula that scales a student's RP balance, relative to the top-performing student in the class, into a bonus internal mark capped at 11. This keeps the incentive fair, visible, and consistent.

2.4 Verifiable, Audited Workflow

Every achievement claim goes through a submit → review → approve/reject workflow with a proof link (GitHub repository, certificate URL, or document), so RP is only credited once faculty or admin have verified the evidence, and every query raised by a student is timestamped and logged until resolved.

2.5 Motivation Through Visibility

A live leaderboard, a weekly RP progress chart, and a class-average comparison give students constant, gamified feedback on where they stand and what activities are available, which is intended to drive sustained participation rather than a last-minute rush before grading.

3. Implementation

3.1 Technology Stack

Layer	Technologies
Frontend	React.js, Vite, Vanilla CSS Design System, Lucide React Icons, Canvas / Print PDF Engine
Backend	Node.js, Express.js, REST APIs, JWT Authentication, Role-Based Access Control (RBAC)
Database	MySQL, with automatic fallback to a JSON-based data engine
Data Import / Export	SheetJS (xlsx) for bulk import, HTML print-to-PDF engine for report cards

3.2 System Architecture & Roles

The application follows a standard three-tier architecture: a React/Vite single-page frontend consumes a REST API exposed by an Express/Node.js backend, which in turn persists data to MySQL (with an automatic JSON-file fallback so the system remains usable without a configured database). Authentication and authorization are handled with JWTs and role-based access control, so each of the three roles below only sees the API endpoints and UI relevant to it.

Role	Responsibilities
Admin	Institution-wide control: bulk RP import, faculty & student provisioning, query audit log, document verification, class-wide performance reports.
Faculty — PS Lead	Problem-solving assessments, LeetCode milestones, lab coding activities.
Faculty — Hackathon Lead	Technical competitions, coding hackathons, paper presentations.
Faculty — Certifications Lead	NPTEL courses, AWS / Cloud certifications, industrial internships.
Student	Tracks RP balance, submits proof of achievement, views rank/leaderboard, downloads report card, raises helpdesk queries.

3.3 Key Functional Modules

Module	Core Function
Excel/CSV Bulk RP Import	Admin downloads a template, uploads a spreadsheet of Email / Purpose / RP columns, previews parsed rows, and credits points to many students in one action.
Query Review & Audit Log	A timestamped log of every student query and staff response, giving a traceable support and grievance-redressal trail.
Activity Posting	Faculty publish RP-bearing activities (title, description, point value) that appear on every student's dashboard under that domain.
RP Allocation / Deduction	Faculty award or deduct points against a student and an optional linked activity, with a memo, updating a live ledger.
Document Proof Submission & Approval	Students submit proof links (GitHub, certificates, Drive); faculty/admin approve or reject, and approval auto-credits RP.
Leaderboard & Ranking	Institution-wide ranking by RP balance, with medal highlights for the top three students.
Internal Marks Engine	Converts RP balance into a bonus internal mark (out of 11) using a proportional scaling formula.
PDF Report Card Generator	Produces a printable, official academic report card per student via a browser print-to-PDF engine.

3.4 Internal Mark Scaling Logic

Reward Points are converted into a bonus internal mark, capped at 11, using a proportional scaling formula:

$$RP \text{ Bonus Marks} = \min(11, \text{round}((\text{Current RP Balance} / \text{Max Class RP Balance}) \times 11, 1))$$

A student's standing is additionally classified for quick reference on the dashboard:

- Good Standing — RP balance at or above the class average RP.

- Medium Standing — RP balance at or above 80% of the class average RP.
- Low Standing — RP balance below 80% of the class average RP.

3.5 Deployment & Setup

The backend runs as an Express API on port 5000 (npm install, npm start inside /backend), and the frontend runs as a Vite dev server or production build (npm install, npm run dev or npm run build inside /frontend). MySQL is optional: the system falls back to a JSON-based data engine and seeders, which lets it be evaluated quickly on a judge's laptop without a database setup step — a practical decision for a time-boxed hackathon build.

3.6 Seeded Demo Accounts

To make the system immediately demonstrable, the repository ships with seeded test accounts spanning all three roles and all three faculty domains:

Role	Email ID	Password	Notes
Admin	admin@college.edu	admin123	Master System Admin
Faculty	faculty.ps@college.edu	faculty123	PS Assessment Lead
Faculty	faculty.hackathon@college.edu	faculty123	Hackathons & Competitions Lead
Faculty	faculty.cert@college.edu	faculty123	Certifications & Internships Lead
Student	s10@college.edu	student123	Jai Deshmukh (Roll: S10)
Student	ukash@bit.ac.in	student123	Ukash D (Roll: UKASH)
Student	vetri@bit.ac.in	student123	Vetriagilan J (Roll: VETRI)

4. Results & Outcomes

Within the hackathon timeframe, the team delivered a working, end-to-end platform rather than a partial prototype. Concretely, the build achieves:

1. A complete role-based system with three fully separated dashboards (Admin, Faculty, Student), each exposing only the actions relevant to that role.
2. An automated bulk-import pipeline that lets an admin credit reward points to an entire class from a single spreadsheet, replacing what would otherwise be dozens of manual entries.
3. A verifiable proof-submission workflow (submit → review → approve/reject) that gives every RP credit a documented, timestamped origin — closing the audit gap in the manual process it replaces.
4. A single, transparent formula that converts RP into internal marks, removing subjective, faculty-by-faculty judgment calls from the scoring process.
5. Real-time motivational tooling — a ranked leaderboard, weekly progress charting, and class-average comparisons — designed to sustain engagement across the semester rather than only at grading time.
6. A functioning helpdesk/query module with response-time tracking, giving students a channel to dispute or ask about points with accountability on the staff side.
7. A one-click, printable PDF report card per student that consolidates RP ledger, rank, and internal marks into an official-looking document, reducing manual report generation for faculty.

8. A deployment path that works with or without MySQL, lowering the barrier for demoing or adopting the system in resource-constrained settings.

4.1 Outcome for Stakeholders

For administrators, the system centralizes institution-wide performance data and removes hours of manual spreadsheet reconciliation. For faculty domain leads, it narrows their evaluation scope to their own specialty while still contributing to a unified class ranking. For students, it converts a previously invisible, faculty-held record into a personal, real-time, gamified dashboard — giving them a clear incentive structure and a transparent way to see exactly how their effort translates into academic credit.

5. Conclusion & Future Scope

The Academic Reward Points & Student Performance Management System demonstrates a practical, deployable answer to a real institutional gap: co-curricular achievement is common, but tracking and rewarding it consistently is not. By combining domain-specific faculty ownership, an auditable submission workflow, and a transparent scoring formula in one full-stack application, the project turns an informal and inconsistent process into a structured, motivating system for students.

Natural next steps beyond the hackathon build would include: automated notifications (email/SMS) on RP allocation or query resolution; deeper analytics for faculty on activity participation trends; integration with external verification sources (e.g. LeetCode or NPTEL APIs) to reduce manual proof review; and a mobile-friendly or native app version of the student dashboard.

Source: github.com/Ukash-07/Doom_Developers_TechNova_Hackathon